University of Central Punjab

Agent Based Modeling, Assignment 1 (group)

Smart Food Delivery Agent

Seven agent architectures for a rider delivering from
Cheezious, Johar Town to the University of Central Punjab

Member	Registration No.
Ch Hamza Younas	L1F23BSAI0089
Rida	L1F23BSAI0091
Muhammad Abdullah	L1F23BSAI0063

Section: _____

Instructor: _____

Date: October 2026

1. Introduction

Food delivery riders in Lahore make the same decision many times a day: which road to take from the restaurant to the customer. The fastest road depends on the time of day, the weather and things the rider cannot see from the restaurant door. This report models one such rider as an intelligent agent. An order is picked up at Cheezious, Johar Town and must be handed over at the main gate of the University of Central Punjab (UCP), Khayaban-e-Jinnah, within 20 minutes.

The same delivery problem is solved by seven single-agent architectures: Simple Reflex, Model-Based Reflex, Goal-Based, Utility-Based, Learning, Hierarchical and Contextual. Every agent receives exactly the same percept and is tested in the same three situations. A Python program produces the results, and an interactive 3D simulation shows each delivery as a rider riding through a model of Johar Town.

2. The Environment

2.1 Road map and routes

There are three ways from Cheezious to UCP. Destinations A, B and C are the three routes:

Route	Road	Distance	Free-flow time	Main risk
A	Khayaban-e-Jinnah Road	3.00 km	10.0 min	Jams at the Jinnah signal chowk in rush hour
B	Canal Bank Road (underpass)	3.25 km	10.4 min	The Canal underpass floods after about 30 minutes of rain
C	Johar Town inner streets	4.20 km	14.4 min	Longest route, but rarely blocked

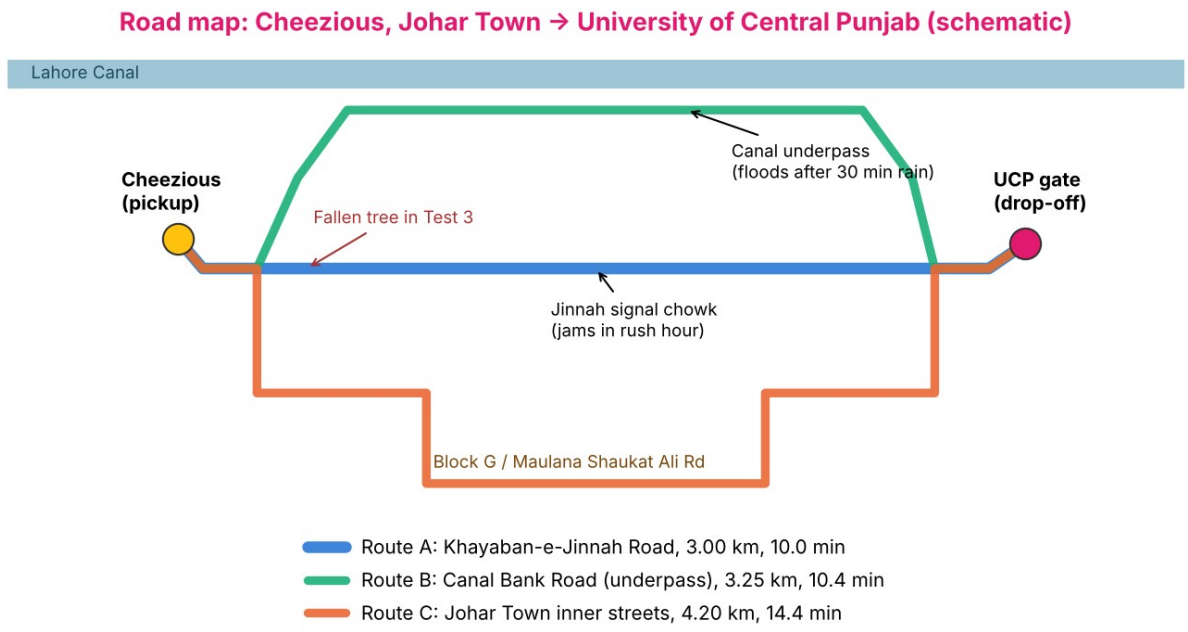


Figure 1. Schematic road map (not to scale).

The road map is a graph with 7 nodes (Cheezious, Jinnah signal chowk, Canal Bank Road entry, Canal underpass, Block G inner street, Maulana Shaukat Ali Road, UCP gate) and 8 road sections, each with a length in kilometres. Riding time is distance divided by the route's average speed. A jam adds a fixed

delay (9.2 min on A), and a flooded underpass adds 14.1 min to B, where the rider has to push the bike through the water.

2.2 PEAS description

	Description
Performance	Order handed over within 20 minutes; shortest delivery time; avoid unsafe (flooded) roads; arrive at all (do not choose a closed road).
Environment	Johar Town roads between Cheezious and UCP: routes A, B, C, traffic, rain, road closures, underpass flooding, time of day.
Actuators	Motorbike controls: choose and ride route A, B or C; speed and braking (used by the Hierarchical agent's operational layer).
Sensors	Eyes (closures and traffic right outside the restaurant, weather), the rider app (clock, live traffic on all routes, reported closures), memory of earlier percepts.

2.3 Properties of the task environment

Property	Value	Reason
Observable	Partially	Flooding of the underpass is 2 km away and is not reported by the app.
Agents	Single agent	Other traffic is part of the environment, not competing agents.
Deterministic	Stochastic (to the agent)	Whether the underpass is flooded is uncertain given the percept; the Learning agent's training trips have random noise.
Episodic	Episodic	Each delivery is independent; the Learning agent uses earlier episodes only for training.
Static	Static during the decision	The world does not change while the agent decides; the route is chosen once at the restaurant.
Discrete	Discrete	Three actions (A, B, C) and a finite set of road conditions.
Known	Known	The rider knows the road map and the rules of the environment.

2.4 Percept

Every agent receives the same percept. Example from Test 3:

```
time:     "13:00"
weather:  "rain"
traffic:  {"A": "light", "B": "light", "C": "light"}
closed:   {"A": true, "B": false, "C": false}
deadline: 20.0
history:  [{"time": "12:00", "weather": "clear"}, {"time": "12:15", "weather": "rain"},
{"time": "12:30", "weather": "rain"}, {"time": "12:45", "weather": "rain"}]
```

3. Test situations

Test	Situation	Order time	What is true	What the rider can see
1	Normal lunch hour	13:00	All roads clear	Everything
2	Evening rush hour	18:10	A jammed at Jinnah chowk (+9.2 min)	A heavy on the traffic app
3	Monsoon rain	13:00	Rain since 12:15; A closed by a fallen tree; B underpass flooded (+14.1 min)	Rain, A closed. Flooding is NOT visible

The best possible choice is A (10.0 min) in Test 1, B (10.4 min) in Test 2 and C (14.4 min) in Test 3.

4. Agent designs

4.1 Simple Reflex Agent

Uses condition-action rules on the current percept only. It has no memory and no model of the world.

```
IF A is closed THEN take B
IF A is jammed THEN take B
OTHERWISE take A
```

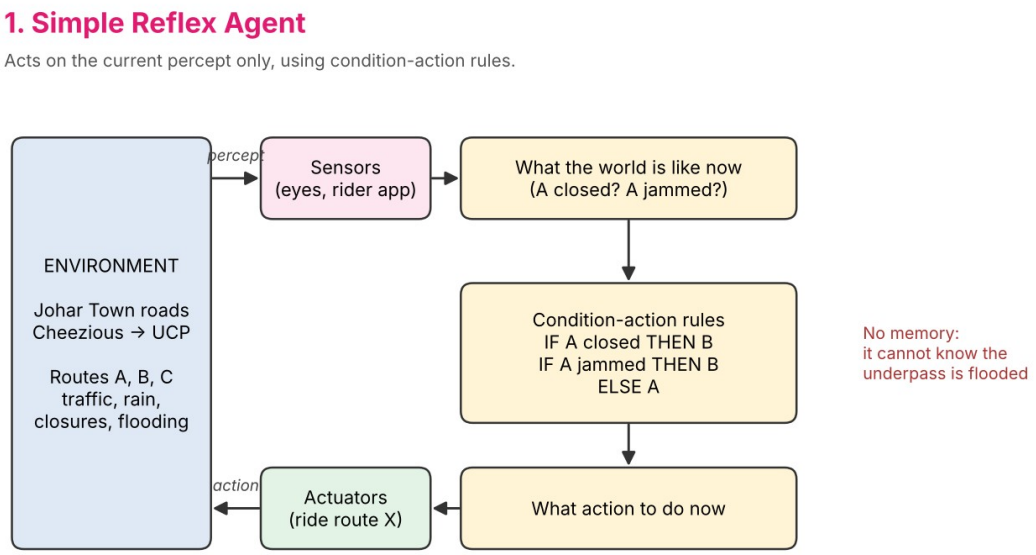


Figure: Simple Reflex Agent architecture.

Fast and simple. In Test 3 it cannot know that the underpass is flooded, so it sends the rider into the water.

Python code

The complete class from code/agents.py:

```
class SimpleReflexAgent(Agent):
    name = "Simple Reflex"
    key = "simple_reflex"

    RULES = [
        ("Khayaban-e-Jinnah is closed", lambda p: p["closed"]["A"], "B"),
        ("Khayaban-e-Jinnah is jammed", lambda p: p["traffic"]["A"] == "heavy", "B"),
        ("the road ahead is clear", lambda p: True, "A"),
    ]

    def decide(self, percept):
        for condition, test, action in self.RULES:
            if test(percept):
                return action, ["Rule fired: IF %s THEN take route %s." % (condition, action),
                                "No memory: earlier percepts are ignored."]
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
```

```
agent = SimpleReflexAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Rule fired: IF the road ahead is clear THEN take route A.
No memory: earlier percepts are ignored.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route B, 10.4 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = SimpleReflexAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Rule fired: IF Khayaban-e-Jinnah is jammed THEN take route B.
No memory: earlier percepts are ignored.
>>> outcome
{'route': 'B', 'minutes': 10.4, 'on_time': True, 'note': 'smooth ride'}
```

Test 3, Monsoon rain: route B, 24.5 min, LATE by 4.5 min.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = SimpleReflexAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Rule fired: IF Khayaban-e-Jinnah is closed THEN take route B.
No memory: earlier percepts are ignored.
>>> outcome
{'route': 'B', 'minutes': 24.5, 'on_time': False, 'note': 'waded through flooded underpass'}
```

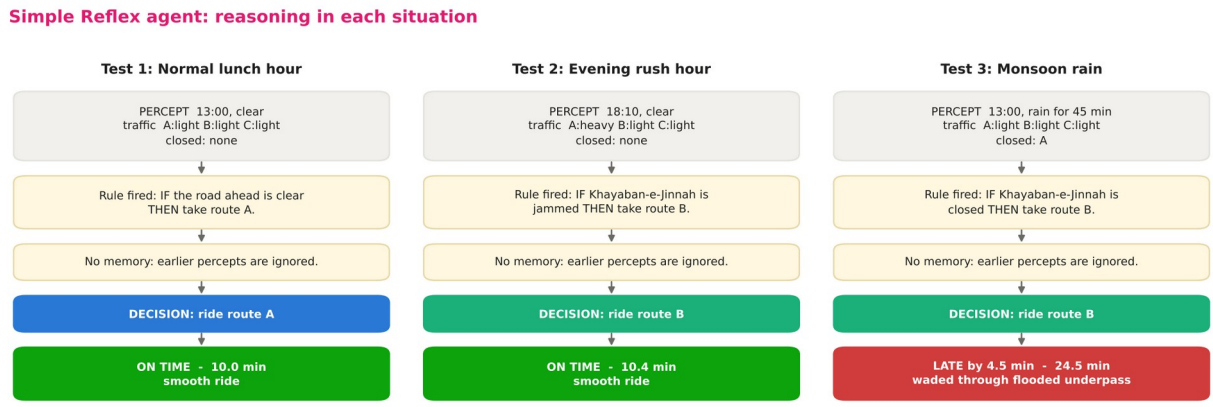


Figure: Simple Reflex Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

4.2 Model-Based Reflex Agent

Keeps an internal state that it updates from the percept history, using a model of how the world works: the Canal underpass floods after 30 minutes of continuous rain.

```
state.rain_minutes = time since rain started (from percept history)
state.B_flooded = rain_minutes >= 30
IF A not blocked THEN A ELSE IF B not flooded THEN B ELSE C
```

2. Model-Based Reflex Agent

Keeps an internal state, updated from the percept history and a model of how the world works.

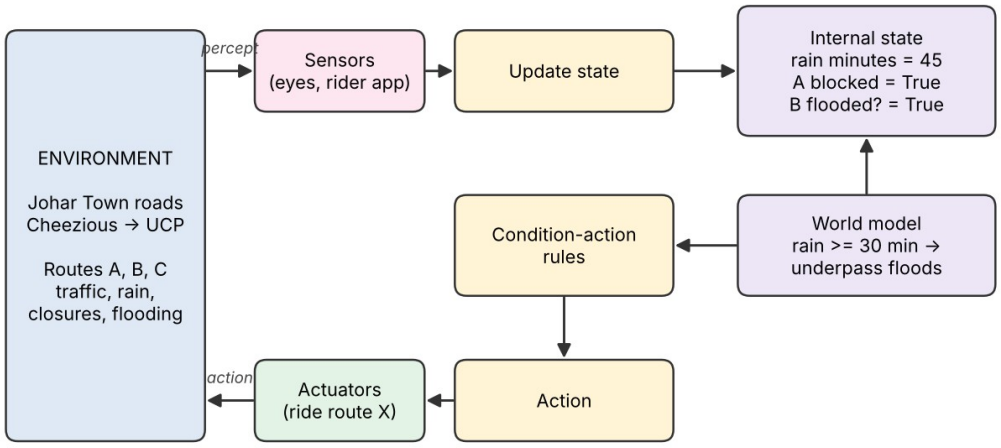


Figure: Model-Based Reflex Agent architecture.

Handles the partially observable flood by remembering how long it has been raining.

Python code

The complete class from code/agents.py:

```
class ModelBasedReflexAgent(Agent):
    name = "Model-Based Reflex"
    key = "model_based"

    def reset(self):
        self.state = {}

    def update_state(self, percept):
        rain = rain_minutes(percept)
        self.state = {
            "rain_minutes": rain,
            "A_blocked": percept["closed"]["A"] or percept["traffic"]["A"] == "heavy",
            # world model: the canal underpass floods after long rain
            "B_flooded": rain >= FLOOD_RAIN_MIN,
            "B_blocked": percept["closed"]["B"] or percept["traffic"]["B"] == "heavy",
        }

    def decide(self, percept):
        self.update_state(percept)
        s = self.state
        why = ["Internal state: raining for %d min, A blocked=%s, underpass flooded (believed)=%s."
               % (s["rain_minutes"], s["A_blocked"], s["B_flooded"])]
        if not s["A_blocked"]:
            route = "A"
            why.append("Rule: main road is fine -> take A.")
        elif not s["B_flooded"] and not s["B_blocked"]:
            route = "B"
            why.append("Rule: A blocked and canal road believed dry -> take B.")
        else:
            route = "C"
            why.append("Rule: A blocked and underpass believed flooded -> take C.")
        return route, why
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
agent = ModelBasedReflexAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Internal state: raining for 0 min, A blocked=False, underpass flooded (believed)=False.
Rule: main road is fine -> take A.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route B, 10.4 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = ModelBasedReflexAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Internal state: raining for 0 min, A blocked=True, underpass flooded (believed)=False.
Rule: A blocked and canal road believed dry -> take B.
>>> outcome
{'route': 'B', 'minutes': 10.4, 'on_time': True, 'note': 'smooth ride'}
```

Test 3, Monsoon rain: route C, 14.4 min, on time.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = ModelBasedReflexAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'C'
>>> print(*why, sep='\n')
Internal state: raining for 45 min, A blocked=True, underpass flooded (believed)=True.
Rule: A blocked and underpass believed flooded -> take C.
>>> outcome
{'route': 'C', 'minutes': 14.4, 'on_time': True, 'note': 'smooth ride'}
```

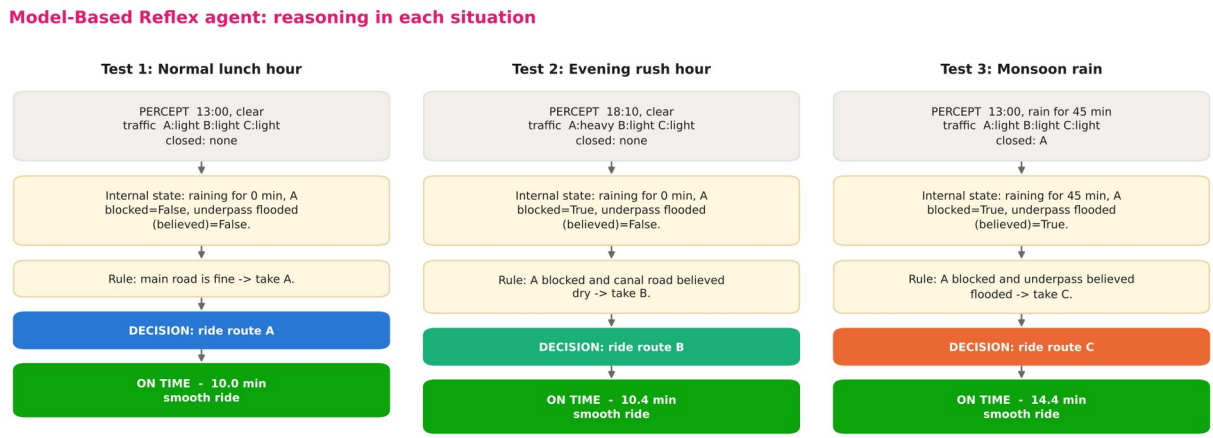


Figure: Model-Based Reflex Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

4.3 Goal-Based Agent

Has an explicit goal (be at the UCP gate) and searches the road graph with uniform-cost search, using distance as the cost. Closed roads are removed from the graph.

```
graph = road map without closed roads
path = uniform_cost_search(graph, CHZ, UCP)
ride the route of path
```

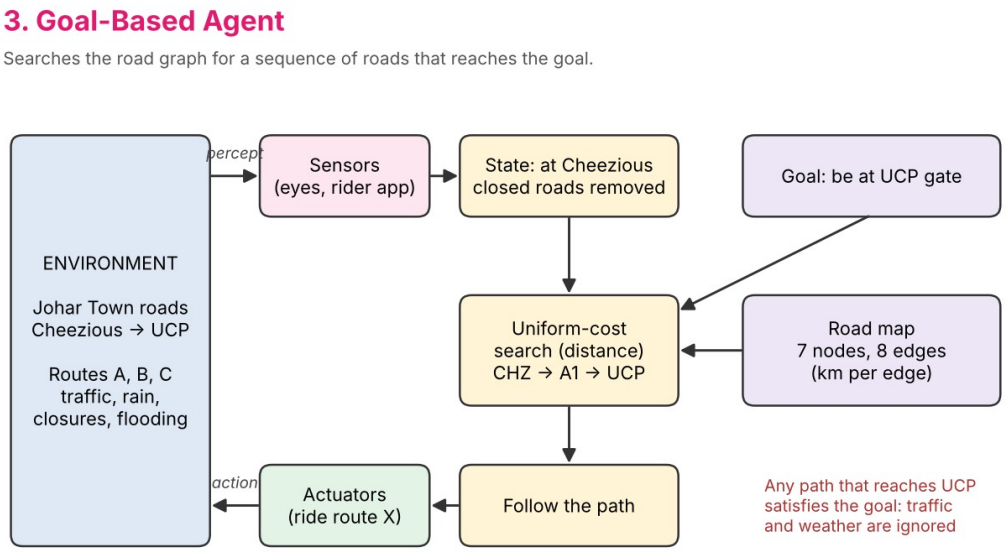


Figure: Goal-Based Agent architecture.

Always reaches the goal by the shortest road, but the goal test does not care about traffic or weather. This costs time in Test 2 and makes it late in Test 3.

Python code

The complete class from code/agents.py:

```
class GoalBasedAgent(Agent):
    name = "Goal-Based"
    key = "goal_based"

    def search(self, percept):
```

```
        graph = {}
        for a, b, km, route in EDGES:
            if percept["closed"][route]:
                continue
            graph.setdefault(a, []).append((b, km, route))
        frontier = [(0.0, "CHZ", [], None)]
        seen = set()
        while frontier:
            cost, node, path, route = heapq.heappop(frontier)
            if node == "UCP":
                return route, path, round(cost, 2)
            if node in seen:
                continue
            seen.add(node)
            for nxt, km, r in graph.get(node, []):
                if nxt not in seen:
                    heapq.heappush(frontier, (cost + km, nxt, path + [nxt], r))
        return None, [], None

    def decide(self, percept):
        route, path, km = self.search(percept)
        closed = [r for r in ROUTE_IDS if percept["closed"][r]]
        why = ["Goal: reach UCP. Closed roads removed from the map: %s."
               % (", ".join(closed) if closed else "none"),
               "Uniform-cost search found CHZ -> %s (%.2f km) on route %s."
               % (" -> ".join(path), km, route),
               "Traffic and weather are not part of the goal test, so they are ignored."]
        return route, why
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
agent = GoalBasedAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Goal: reach UCP. Closed roads removed from the map: none.
Uniform-cost search found CHZ -> A1 -> UCP (3.00 km) on route A.
Traffic and weather are not part of the goal test, so they are ignored.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route A, 19.2 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = GoalBasedAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Goal: reach UCP. Closed roads removed from the map: none.
Uniform-cost search found CHZ -> A1 -> UCP (3.00 km) on route A.
Traffic and weather are not part of the goal test, so they are ignored.
>>> outcome
{'route': 'A', 'minutes': 19.2, 'on_time': True, 'note': 'stuck in traffic at signal chowk queue'}
```

Test 3, Monsoon rain: route B, 24.5 min, LATE by 4.5 min.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = GoalBasedAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Goal: reach UCP. Closed roads removed from the map: A.
Uniform-cost search found CHZ -> B1 -> B2 -> UCP (3.25 km) on route B.
Traffic and weather are not part of the goal test, so they are ignored.
>>> outcome
{'route': 'B', 'minutes': 24.5, 'on_time': False, 'note': 'waded through flooded underpass'}
```

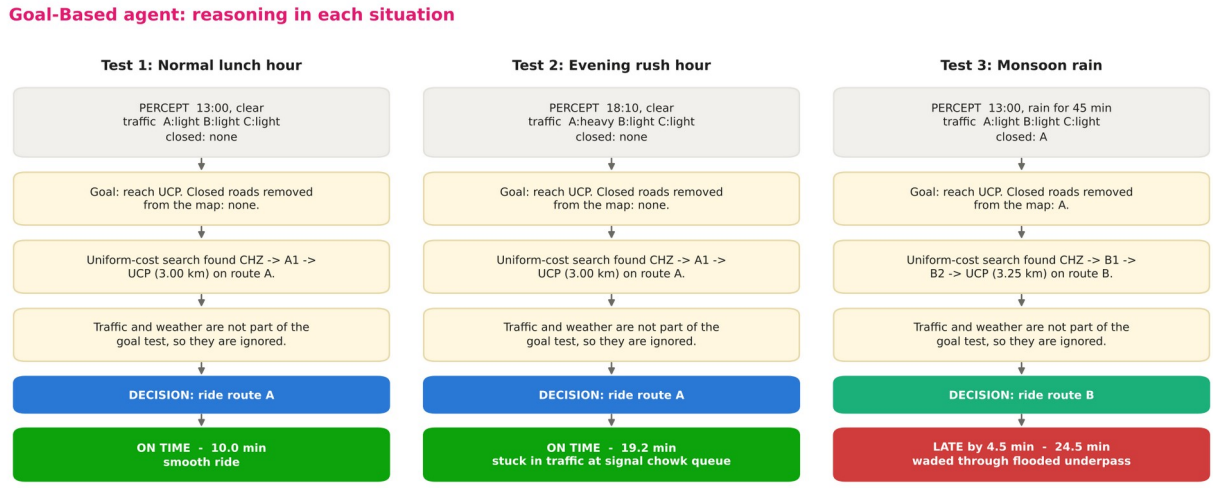


Figure: Goal-Based Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

4.4 Utility-Based Agent

Predicts the possible outcomes of each route with probabilities, for example P(flood) = 0.85 after more than 30 minutes of rain, and scores them with a utility function:

```
U(route) = 100 x P(on time) - 2 x E[minutes] - 15 x P(flood)
choose the open route with the highest U
```

4. Utility-Based Agent

Predicts the outcomes of every route and picks the highest expected utility.

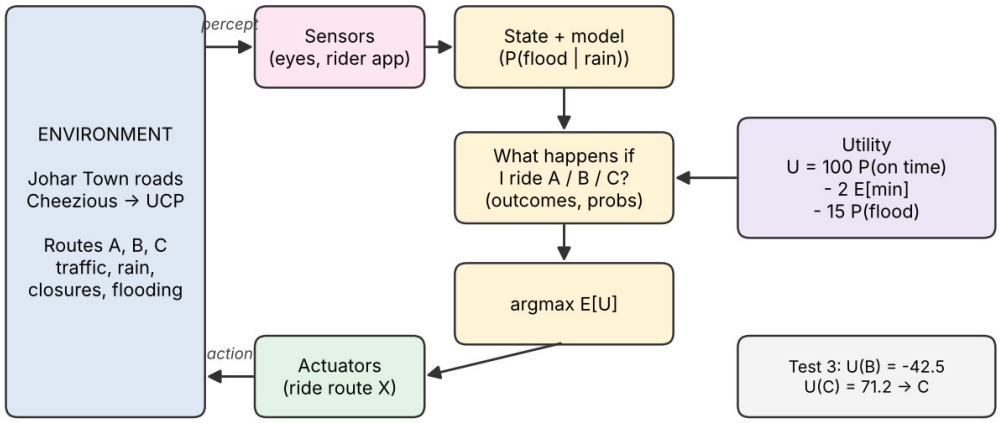


Figure: Utility-Based Agent architecture.

Trades off speed, punctuality and safety, and handles uncertainty explicitly.

Python code

The complete class from code/agents.py:

```
class UtilityBasedAgent(Agent):
    name = "Utility-Based"
    key = "utility_based"

    W_ON_TIME, W_TIME, W_RISK = 100.0, 2.0, 15.0

    @staticmethod
    def flood_probability(rain):
        if rain == 0:
            return 0.0
        return 0.85 if rain >= FLOOD_RAIN_MIN else 0.30

    def outcomes(self, route, percept):
        """List of (probability, minutes) for a route."""
        t = base_minutes(route)
        if percept["traffic"][route] == "heavy":
            t += ROUTES[route]["jam_delay"]
        p = self.flood_probability(rain_minutes(percept)) if ROUTES[route]["flood_delay"] else 0.0
        if p == 0.0:
            return [(1.0, round(t, 1))], 0.0
        return [(1 - p, round(t, 1)), (p, round(t + ROUTES[route]["flood_delay"], 1))], p

    def utility(self, route, percept):
        outs, risk = self.outcomes(route, percept)
        expected = sum(p * t for p, t in outs)
        p_on_time = sum(p for p, t in outs if t <= DEADLINE_MIN)
        u = self.W_ON_TIME * p_on_time - self.W_TIME * expected - self.W_RISK * risk
        return round(u, 2), round(expected, 2), round(p_on_time, 2)

    def decide(self, percept):
        why = ["U = 100*P(on time) - 2*E[minutes] - 15*P(flood)"]
        best, best_u = None, None
        for r in open_routes(percept):
            u, e, pot = self.utility(r, percept)
            why.append("Route %s: E[t]=%.1f min, P(on time)=%.2f, U=%.1f" % (r, e, pot, u))
            if best_u is None or u > best_u:
                best, best_u = r, u
        why.append("Highest utility -> route %s." % best)
        return best, why
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
agent = UtilityBasedAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
U = 100*P(on time) - 2*E[minutes] - 15*P(flood)
Route A: E[t]=10.0 min, P(on time)=1.00, U=80.0
Route B: E[t]=10.4 min, P(on time)=1.00, U=79.2
Route C: E[t]=14.4 min, P(on time)=1.00, U=71.2
Highest utility -> route A.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route B, 10.4 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = UtilityBasedAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
U = 100*P(on time) - 2*E[minutes] - 15*P(flood)
Route A: E[t]=19.2 min, P(on time)=1.00, U=61.6
Route B: E[t]=10.4 min, P(on time)=1.00, U=79.2
Route C: E[t]=14.4 min, P(on time)=1.00, U=71.2
Highest utility -> route B.
>>> outcome
{'route': 'B', 'minutes': 10.4, 'on_time': True, 'note': 'smooth ride'}
```

Test 3, Monsoon rain: route C, 14.4 min, on time.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = UtilityBasedAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'C'
>>> print(*why, sep='\n')
U = 100*P(on time) - 2*E[minutes] - 15*P(flood)
Route B: E[t]=22.4 min, P(on time)=0.15, U=-42.5
Route C: E[t]=14.4 min, P(on time)=1.00, U=71.2
Highest utility -> route C.
>>> outcome
{'route': 'C', 'minutes': 14.4, 'on_time': True, 'note': 'smooth ride'}
```



Figure: Utility-Based Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

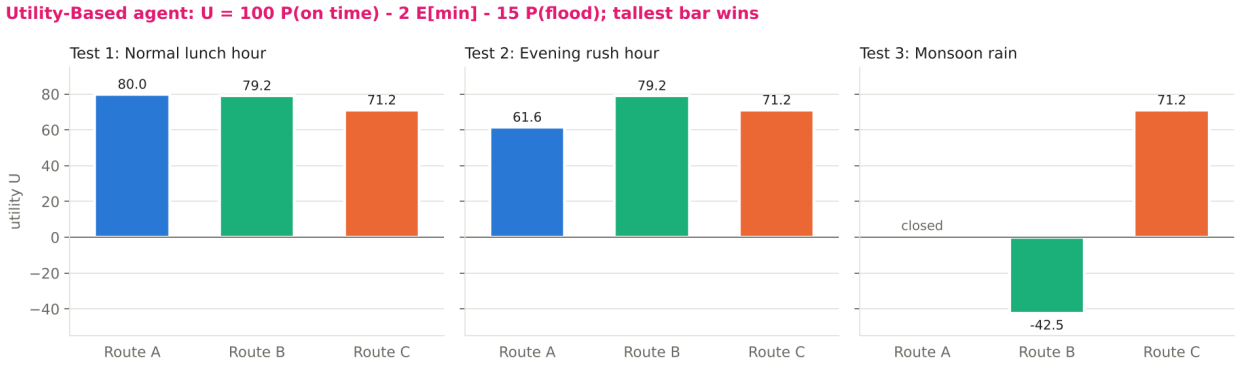


Figure: utility of each route in each test. The agent rides the tallest bar.

4.5 Learning Agent

Learns a value Q(context, route) from experience. During training it rides every route on six practice days three times (54 trips, with a little random noise) and keeps the running average of -minutes. Contexts are normal, rush and rain.

```
Q(ctx, r) <- Q(ctx, r) + (reward - Q(ctx, r)) / n      reward = -minutes
at test time: choose argmax_r Q(ctx, r) among open routes
```

5. Learning Agent

Improves its route values from experience (54 training trips, running-average Q-values).

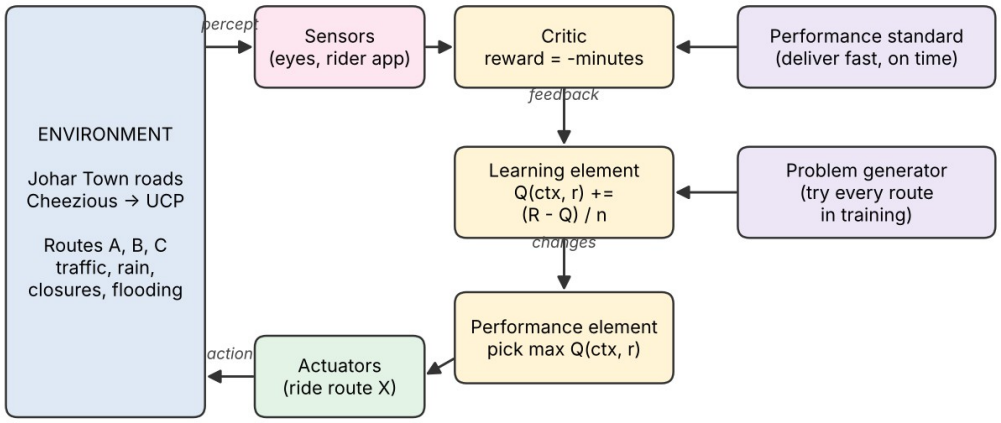


Figure: Learning Agent architecture.

Needs no hand-written rules, but it needs training data that covers each context.

Python code

The complete class from code/agents.py:

```
class LearningAgent(Agent):
    name = "Learning"
    key = "learning"

    ROUNDS = 3

    def __init__(self):
        self.q = {}
        self.n = {}
        self.trained = False

    @staticmethod
    def context(percept):
        if percept["weather"] == "rain":
            return "rain"
        return "rush" if is_rush(percept["time"]) else "normal"

    def train(self):
        """Ride every route on every training day a few times (exploration),
        keeping the running average of -minutes as the Q value."""
        noise = Noise(42)
        self.q, self.n = {}, {}
        for _ in range(self.ROUNDS):
            for day in TRAINING_DAYS:
                ctx = "rain" if day["weather"] == "rain" else ("rush" if is_rush(day["time"]) else
"normal")
                for r in ROUTE_IDS:
                    t = trip_minutes(r, day["state"])
                    if t is None:
                        continue
                    reward = -(t + noise.minutes(1.0))
                    k = (ctx, r)
                    self.n[k] = self.n.get(k, 0) + 1
                    old = self.q.get(k, 0.0)
                    self.q[k] = old + (reward - old) / self.n[k]
        self.trained = True

    def decide(self, percept):
        if not self.trained:
```

```
            self.train()
        ctx = self.context(percept)
        why = ["Context recognised: %s. Learned from %d training trips."
               % (ctx, sum(self.n.values()))]
        best, best_q = None, None
        for r in open_routes(percept):
            q = self.q.get((ctx, r))
            if q is None:
                continue
            why.append("Q(%s, %s) = %.1f  (about %.1f min)" % (ctx, r, q, -q))
            if best_q is None or q > best_q:
                best, best_q = r, q
        why.append("Exploit best learned value -> route %s." % best)
        return best, why
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
agent = LearningAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Context recognised: normal. Learned from 54 training trips.
Q(normal, A) = -9.9  (about 9.9 min)
Q(normal, B) = -10.5  (about 10.5 min)
Q(normal, C) = -14.3  (about 14.3 min)
Exploit best learned value -> route A.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route B, 10.4 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = LearningAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Context recognised: rush. Learned from 54 training trips.
Q(rush, A) = -19.1  (about 19.1 min)
Q(rush, B) = -10.3  (about 10.3 min)
Q(rush, C) = -15.9  (about 15.9 min)
Exploit best learned value -> route B.
>>> outcome
{'route': 'B', 'minutes': 10.4, 'on_time': True, 'note': 'smooth ride'}
```

Test 3, Monsoon rain: route C, 14.4 min, on time.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = LearningAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
```

```
'C'
>>> print(*why, sep='\n')
Context recognised: rain. Learned from 54 training trips.
Q(rain, B) = -24.5  (about 24.5 min)
Q(rain, C) = -14.3  (about 14.3 min)
Exploit best learned value -> route C.
>>> outcome
{'route': 'C', 'minutes': 14.4, 'on_time': True, 'note': 'smooth ride'}
```

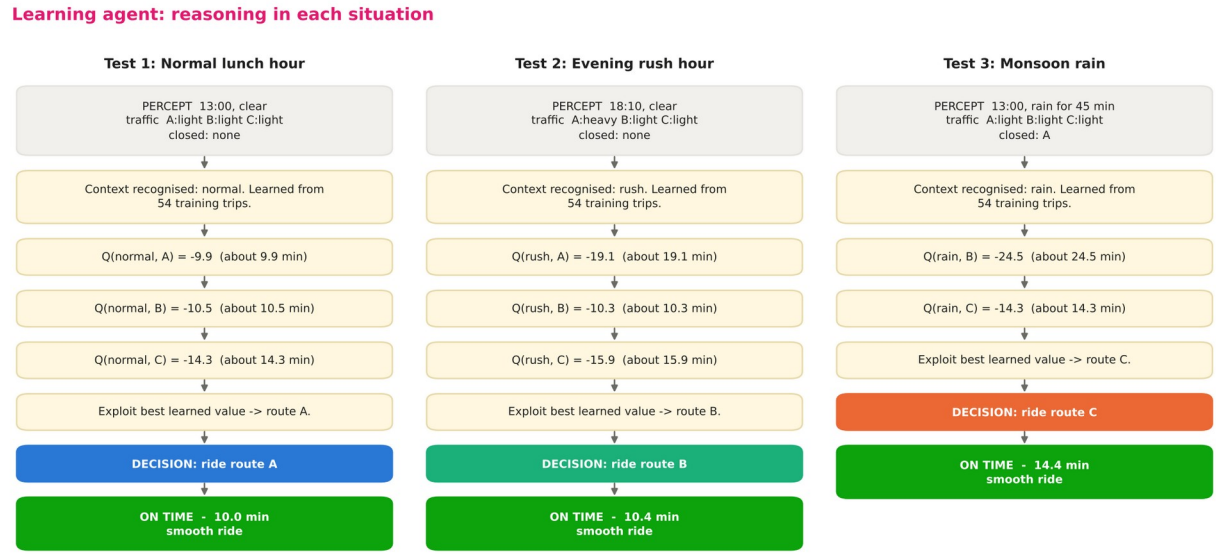


Figure: Learning Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

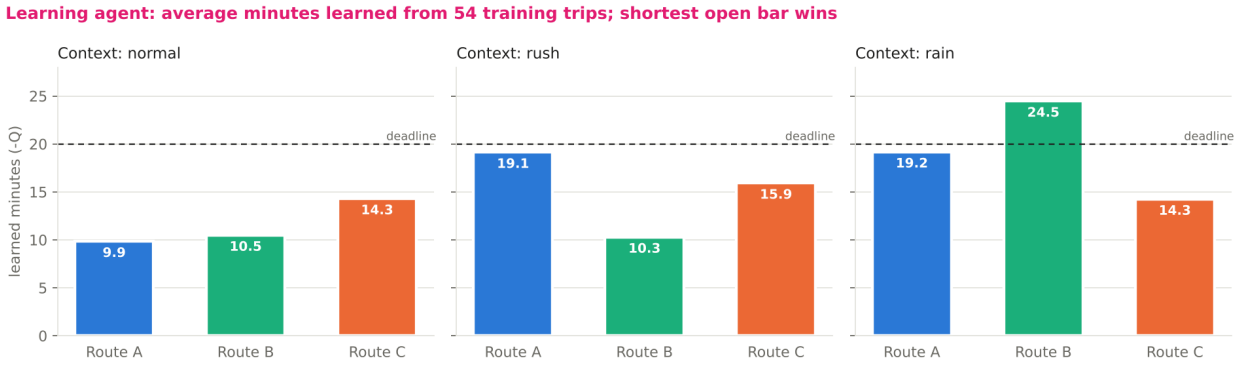


Figure: minutes the Learning agent learned for each route in each context.

4.6 Hierarchical Agent

Splits the problem into three layers. The strategic layer sets the mode (fast or safe) and constraints (avoid congestion, avoid flood-prone roads after 30 minutes of rain). The tactical layer picks the fastest route that meets the constraints. The operational layer controls riding (speed limit, signals).

```
constraints = strategic(percept)
route = fastest route that satisfies constraints
operational(route): ride it
```

6. Hierarchical Agent

Three layers: each one sets the problem for the layer below.

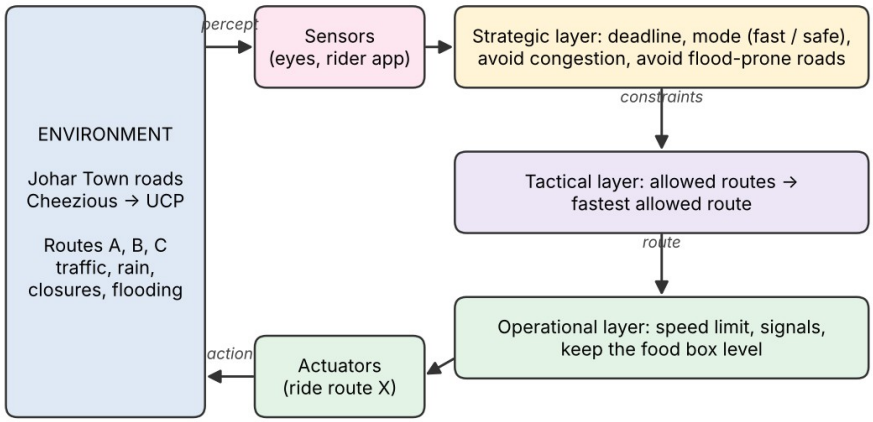


Figure: Hierarchical Agent architecture.

Easy to extend: a new concern (for example fuel) is added in one layer.

Python code

The complete class from code/agents.py:

```
class HierarchicalAgent(Agent):
    name = "Hierarchical"
    key = "hierarchical"

    def strategic(self, percept):
        rain = rain_minutes(percept)
        return {"avoid_congestion": True,
                "avoid_flood_prone": rain >= FLOOD_RAIN_MIN,
                "mode": "safe" if percept["weather"] == "rain" else "fast"}

    def tactical(self, percept, plan):
        candidates = []
        for r in open_routes(percept):
            if plan["avoid_congestion"] and percept["traffic"][r] == "heavy":
                continue
            if plan["avoid_flood_prone"] and ROUTES[r]["flood_delay"] > 0:
                continue
            candidates.append(r)
        if not candidates:
            candidates = open_routes(percept)
        return min(candidates, key=base_minutes), candidates

    def operational(self, route, plan):
        speed = ROUTES[route]["speed_kmh"] * (0.8 if plan["mode"] == "safe" else 1.0)
        return "limit speed to %.0f km/h on bad patches, stop at red signals, keep the food box level"
% speed

    def decide(self, percept):
        plan = self.strategic(percept)
        route, candidates = self.tactical(percept, plan)
        return route, [
            "Strategic layer: mode=%s, avoid congestion=%s, avoid flood-prone=%s."
            % (plan["mode"], plan["avoid_congestion"], plan["avoid_flood_prone"]),
            "Tactical layer: allowed routes %s, fastest of them is %s."
            % (", ".join(candidates), route),
            "Operational layer: %s." % self.operational(route, plan),
        ]
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
agent = HierarchicalAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Strategic layer: mode=fast, avoid congestion=True, avoid flood-prone=False.
Tactical layer: allowed routes A, B, C, fastest of them is A.
Operational layer: limit speed to 18 km/h on bad patches, stop at red signals, keep the food box
level.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route B, 10.4 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = HierarchicalAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Strategic layer: mode=fast, avoid congestion=True, avoid flood-prone=False.
Tactical layer: allowed routes B, C, fastest of them is B.
Operational layer: limit speed to 19 km/h on bad patches, stop at red signals, keep the food box
level.
>>> outcome
{'route': 'B', 'minutes': 10.4, 'on_time': True, 'note': 'smooth ride'}
```

Test 3, Monsoon rain: route C, 14.4 min, on time.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = HierarchicalAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'C'
>>> print(*why, sep='\n')
Strategic layer: mode=safe, avoid congestion=True, avoid flood-prone=True.
Tactical layer: allowed routes C, fastest of them is C.
Operational layer: limit speed to 14 km/h on bad patches, stop at red signals, keep the food box
level.
>>> outcome
{'route': 'C', 'minutes': 14.4, 'on_time': True, 'note': 'smooth ride'}
```

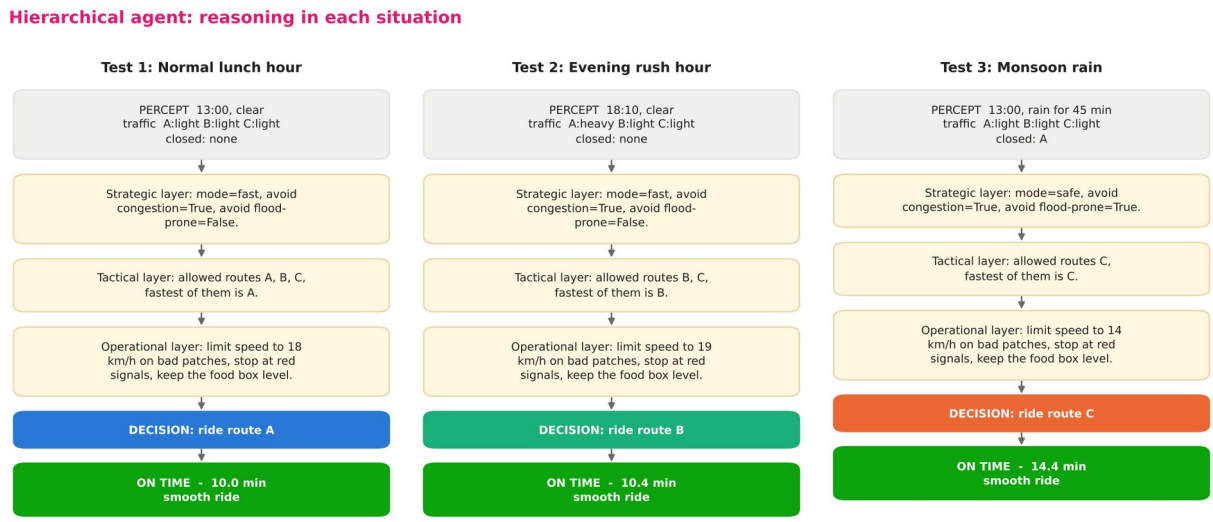


Figure: Hierarchical Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

4.7 Contextual Agent

First recognises the context (normal, rush hour or monsoon) from the clock, weather and traffic, then applies the policy written for that context.

```
normal: A > B > C     rush hour: B > C > A     monsoon: C > A > B
choose the first open route in the policy
```

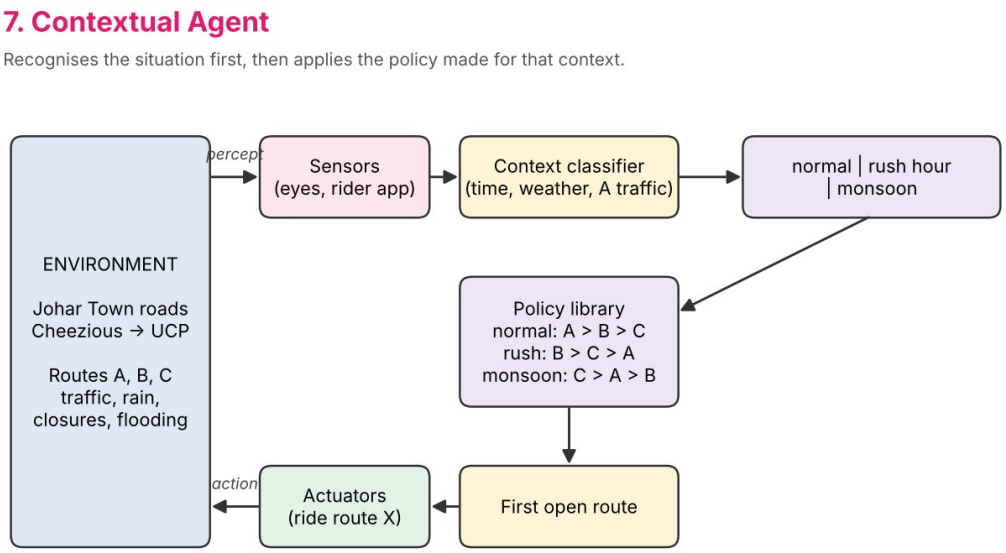


Figure: Contextual Agent architecture.

Very quick decisions, but every context and its policy has to be designed in advance.

Python code

The complete class from code/agents.py:

```
class ContextualAgent(Agent):
    name = "Contextual"
    key = "contextual"

    POLICIES = {
        "normal": ["A", "B", "C"],       # shortest first
```

```
        "rush hour": ["B", "C", "A"],    # keep off the main road
        "monsoon": ["C", "A", "B"],      # keep out of the underpass
    }

    @staticmethod
    def classify(percept):
        if percept["weather"] == "rain":
            return "monsoon"
        if is_rush(percept["time"]) or percept["traffic"]["A"] == "heavy":
            return "rush hour"
        return "normal"

    def decide(self, percept):
        ctx = self.classify(percept)
        order = self.POLICIES[ctx]
        route = next(r for r in order if not percept["closed"][r])
        return route, ["Context: %s (time %s, weather %s)." % (ctx, percept["time"],
percept["weather"]),
                       "Policy for %s: preference %s." % (ctx, " > ".join(order)),
                       "First open route in the policy -> %s." % route]
```

Behaviour in each situation

Each snippet runs the agent on one test situation and shows its real output (decision, reasoning and the delivery result from the environment).

Test 1, Normal lunch hour: route A, 10.0 min, on time.

```
# Test 1: Normal lunch hour
scenario = SCENARIOS[0]
agent = ContextualAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'A'
>>> print(*why, sep='\n')
Context: normal (time 13:00, weather clear).
Policy for normal: preference A > B > C.
First open route in the policy -> A.
>>> outcome
{'route': 'A', 'minutes': 10.0, 'on_time': True, 'note': 'smooth ride'}
```

Test 2, Evening rush hour: route B, 10.4 min, on time.

```
# Test 2: Evening rush hour
scenario = SCENARIOS[1]
agent = ContextualAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)

# output
>>> route
'B'
>>> print(*why, sep='\n')
Context: rush hour (time 18:10, weather clear).
Policy for rush hour: preference B > C > A.
First open route in the policy -> B.
>>> outcome
{'route': 'B', 'minutes': 10.4, 'on_time': True, 'note': 'smooth ride'}
```

Test 3, Monsoon rain: route C, 14.4 min, on time.

```
# Test 3: Monsoon rain
scenario = SCENARIOS[2]
agent = ContextualAgent()
route, why = agent.decide(make_percept(scenario))
outcome = execute(route, scenario)
```

```
# output
>>> route
'C'
>>> print(*why, sep='\n')
Context: monsoon (time 13:00, weather rain).
Policy for monsoon: preference C > A > B.
First open route in the policy -> C.
>>> outcome
{'route': 'C', 'minutes': 14.4, 'on_time': True, 'note': 'smooth ride'}
```

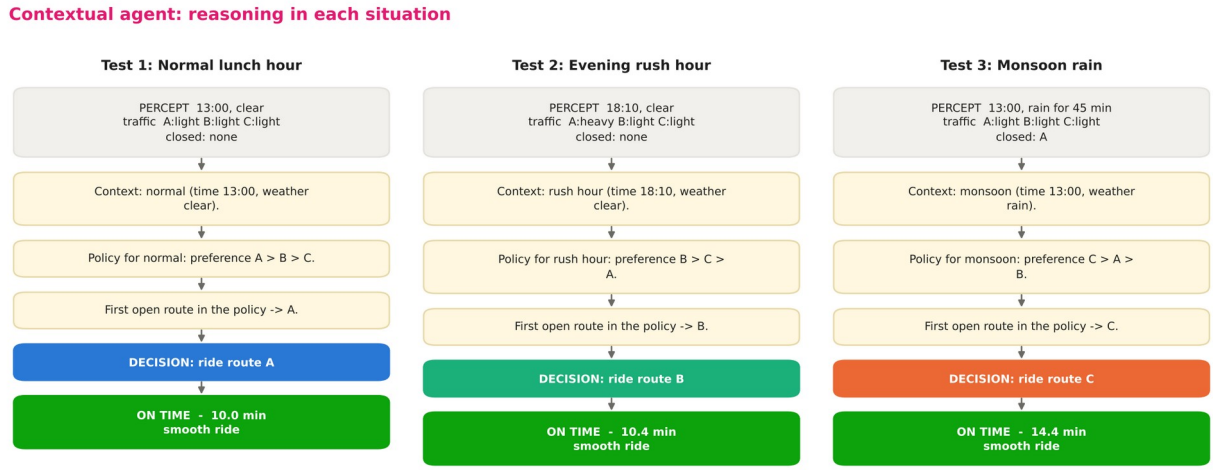


Figure: Contextual Agent agent, reasoning trace in the three situations (percept, every reasoning step, decision, outcome).

5. Implementation

File	Purpose
code/environment.py	Road map, routes, test situations, percept, trip-time model, training days
code/agents.py	The seven agent classes; each has decide(percept) -> (route, reasoning)
code/main.py	Runs every agent on every test, prints the tables, writes results/results.txt and .json
simulation/	Interactive 3D simulation (Three.js): index.html, model.js, world.js, app.js
tools/check_parity.js	Checks that simulation/model.js makes exactly the same decisions, times and reasoning as the Python code
tools/export_diagrams.py	Draws the architecture diagrams
tools/build_report.py	Builds this report

Run the program with Python 3.8 or newer (no extra packages):

```
cd code
python main.py            # result tables
python main.py --details  # tables + every agent's reasoning
```

5.1 The 3D simulation

The simulation is a 3D model of the trip. A rider in a pink jacket on a motorbike with a delivery box leaves Cheezious and rides the chosen route to the UCP gate. The traffic keeps left, as in Pakistan. The scene shows Khayaban-e-Jinnah as a dual carriageway with a signal chowk, the Canal Bank Road along the Lahore Canal with its underpass, the inner streets of Block G, cars, rickshaws, buses and bikes, and lighting for the time of day. Test 2 has an evening sky and a traffic queue at the chowk. Test 3 has rain, a fallen tree across Khayaban-e-Jinnah and a flooded underpass with a stalled car.

- The decisions come from simulation/model.js, a line-by-line copy of the Python agents (verified by tools/check_parity.js).

- The rider's speed follows the model: free-flow speed on open road, crawling through the jam, pushing the bike through the flood. The total time equals the model's delivery time.
- The side panel shows the percept, the hidden truth (underpass flooded or dry) and the agent's reasoning line by line.
- Race mode sends all seven agents out at once. A custom mode lets you set the time, rain, jams, closures and flooding yourself.
- One simulated minute takes two seconds at 1x speed. Speeds of 2x, 4x and 8x and chase, map and orbit cameras are available.



Figure: Test 1, the Utility-Based agent rides route A.



Figure: Test 2, the Goal-Based agent crawls through the jam at Jinnah chowk.

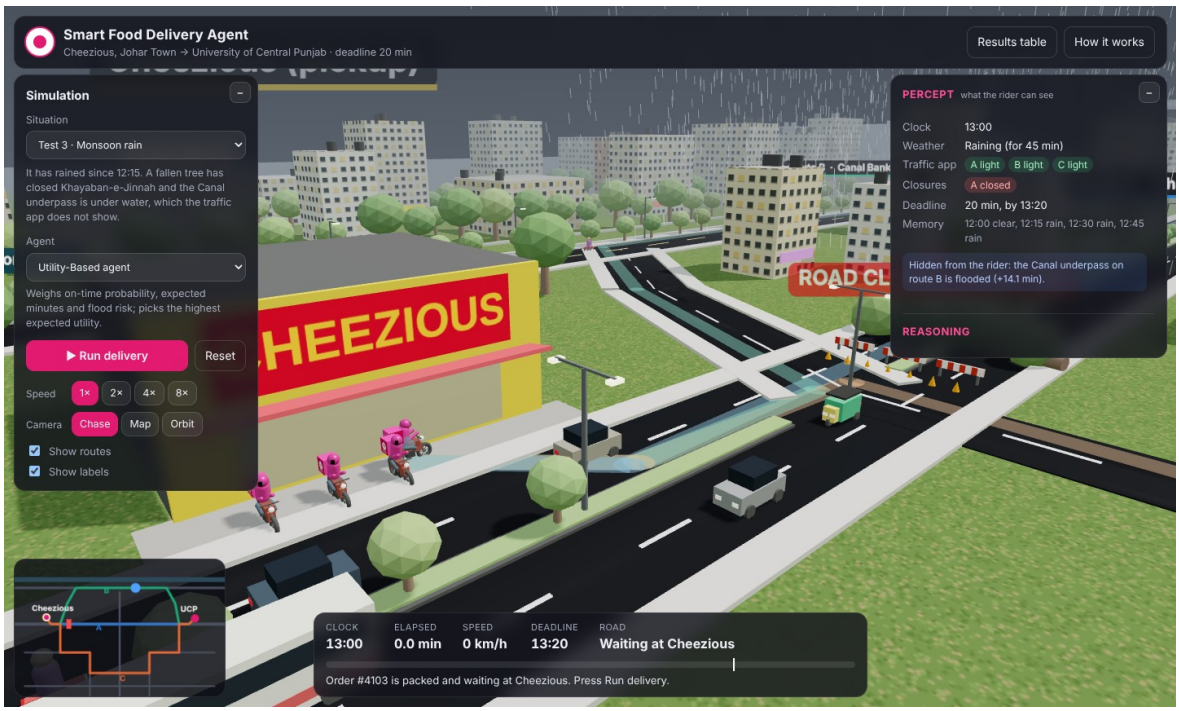


Figure: Test 3, rain, and Khayaban-e-Jinnah closed by a fallen tree.

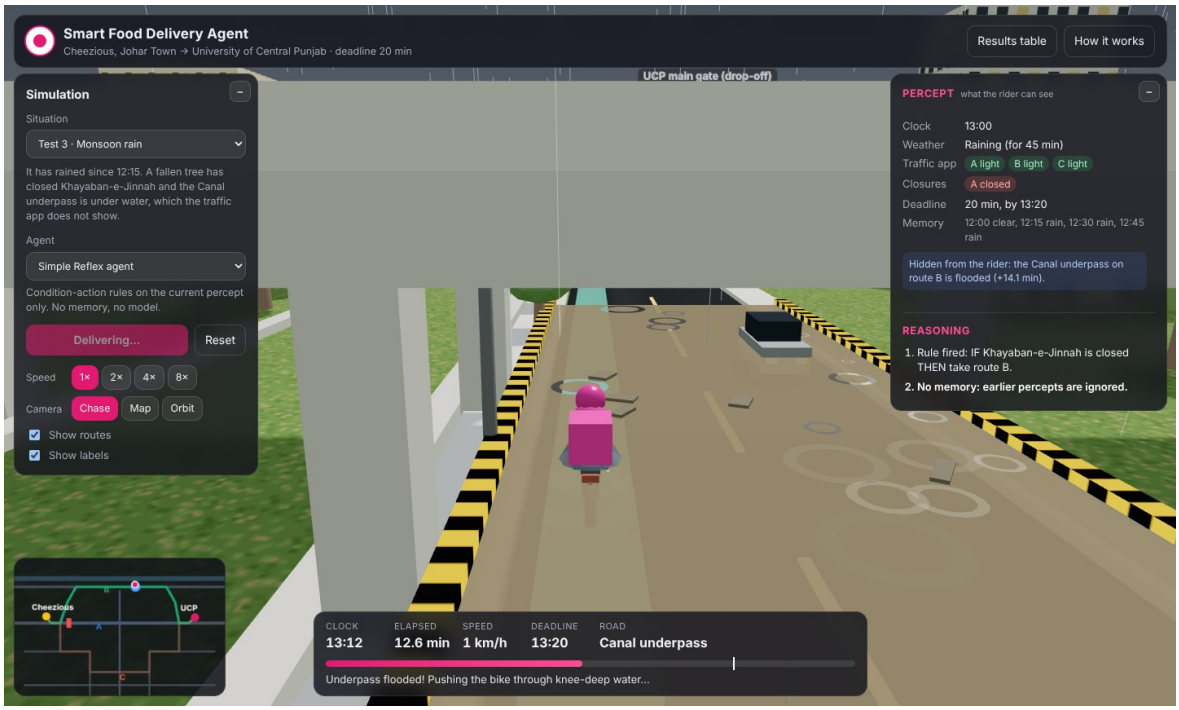


Figure: Test 3, the Simple Reflex agent is stuck in the flooded underpass.



Figure: Test 3 race. Five agents take route C; Simple Reflex and Goal-Based take B.

6. Results

Agent	Test 1	Test 2	Test 3	On time	Total min
Simple Reflex	A 10.0	B 10.4	B 24.5 (late)	2/3	44.9
Model-Based Reflex	A 10.0	B 10.4	C 14.4	3/3	34.8
Goal-Based	A 10.0	A 19.2	B 24.5 (late)	2/3	53.7
Utility-Based	A 10.0	B 10.4	C 14.4	3/3	34.8
Learning	A 10.0	B 10.4	C 14.4	3/3	34.8
Hierarchical	A 10.0	B 10.4	C 14.4	3/3	34.8
Contextual	A 10.0	B 10.4	C 14.4	3/3	34.8

Cell = chosen route and real delivery time in minutes. Deadline: 20 minutes.



Best possible: A 10.0 | B 10.4 | C 14.4

Figure: the route each agent chose in each situation (colour = route).

6.1 Test 1: Normal lunch hour

Clear sky at 1 pm. All three roads are moving normally. Best possible: route A, 10.0 min.

Agent	Route	Min	Result	Margin	vs best	Deciding step
Simple Reflex	A	10.0	On time	10.0 spare	+0.0	Rule fired: IF the road ahead is clear THEN take route A.
Model-Based Reflex	A	10.0	On time	10.0 spare	+0.0	Rule: main road is fine -> take A.
Goal-Based	A	10.0	On time	10.0 spare	+0.0	Uniform-cost search found CHZ -> A1 -> UCP (3.00 km) on route A.
Utility-Based	A	10.0	On time	10.0 spare	+0.0	Highest utility -> route A.
Learning	A	10.0	On time	10.0 spare	+0.0	Exploit best learned value -> route A.
Hierarchical	A	10.0	On time	10.0 spare	+0.0	Tactical layer: allowed routes A, B, C, fastest of them is A.
Contextual	A	10.0	On time	10.0 spare	+0.0	Policy for normal: preference A > B > C.

6.2 Test 2: Evening rush hour

6:10 pm. Office traffic has jammed the Jinnah signal chowk on Khayaban-e-Jinnah. The other roads are clear. Best possible: route B, 10.4 min.

Agent	Route	Min	Result	Margin	vs best	Deciding step
Simple Reflex	B	10.4	On time	9.6 spare	+0.0	Rule fired: IF Khayaban-e-Jinnah is jammed THEN take route B.
Model-Based Reflex	B	10.4	On time	9.6 spare	+0.0	Rule: A blocked and canal road believed dry -> take B.
Goal-Based	A	19.2	On time	0.8 spare	+8.8	Uniform-cost search found CHZ -> A1 -> UCP (3.00 km) on route A.
Utility-Based	B	10.4	On time	9.6 spare	+0.0	Highest utility -> route B.
Learning	B	10.4	On time	9.6 spare	+0.0	Exploit best learned value -> route B.
Hierarchical	B	10.4	On time	9.6 spare	+0.0	Tactical layer: allowed routes B, C, fastest of them is B.
Contextual	B	10.4	On time	9.6 spare	+0.0	Policy for rush hour: preference B > C > A.

6.3 Test 3: Monsoon rain

It has rained since 12:15. A fallen tree has closed Khayaban-e-Jinnah and the Canal underpass is under water, which the traffic app does not show. Best possible: route C, 14.4 min.

Agent	Route	Min	Result	Margin	vs best	Deciding step
Simple Reflex	B	24.5	LATE	4.5 late	+10.1	Rule fired: IF Khayaban-e-Jinnah is closed THEN take route B.
Model-Based Reflex	C	14.4	On time	5.6 spare	+0.0	Rule: A blocked and underpass believed flooded -> take C.
Goal-Based	B	24.5	LATE	4.5 late	+10.1	Uniform-cost search found CHZ -> B1 -> B2 -> UCP (3.25 km) on route B.
Utility-Based	C	14.4	On time	5.6 spare	+0.0	Highest utility -> route C.
Learning	C	14.4	On time	5.6 spare	+0.0	Exploit best learned value -> route C.
Hierarchical	C	14.4	On time	5.6 spare	+0.0	Tactical layer: allowed routes C, fastest of them is C.
Contextual	C	14.4	On time	5.6 spare	+0.0	Policy for monsoon: preference C > A > B.

6.4 Summary of the three situations

- Test 1: every agent takes A (10.0 min). With nothing unusual, all architectures are equal.

- Test 2: the jam on A is visible on the app. Every agent except Goal-Based switches to B (10.4 min). Goal-Based still takes the shortest road A and needs 19.2 min. It is on time, but only just.
- Test 3: A is closed and the underpass on B is flooded, which cannot be seen. Simple Reflex (no memory) and Goal-Based (traffic and weather are not part of its goal) take B and arrive after 24.5 min, which is late. Every agent that keeps or learns a model of the rain (Model-Based, Utility, Learning, Hierarchical, Contextual) takes C and arrives in 14.4 min.

7. Comparison and the most suitable agent

7.1 How the agents behaved differently

All seven agents received exactly the same percepts, so every difference below comes from the architecture alone. Test 1 does not separate them: with nothing unusual, every agent takes the shortest road A. Tests 2 and 3 show how each one reacts when something goes wrong.

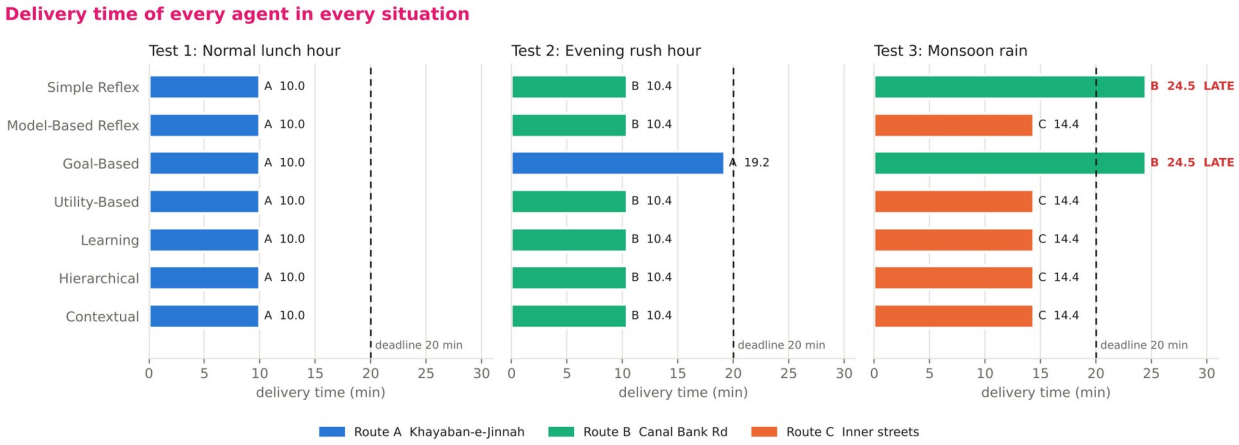


Figure: delivery time per agent and situation; bar colour = route chosen, dashed line = 20-minute deadline.

- Simple Reflex reacts only to what it sees right now. It avoids the visible jam on A in Test 2, but in Test 3 its rule 'A closed, take B' sends the rider into the hidden flood (24.5 min, late).
- Model-Based Reflex uses the same kind of rules, but it remembers that it has been raining for 45 minutes. Its world model says the underpass floods after 30 minutes, so it picks C. Memory is the only difference from Simple Reflex, and it decides Test 3.
- Goal-Based plans a path to UCP on the map, but its cost is distance only. It ignores the jam in Test 2 (19.2 min, on time by only 0.8 min) and the flood in Test 3 (late). Reaching the goal is not the same as reaching it well.
- Utility-Based compares the expected outcome of every route. In Test 3 it gives B a utility of -42.5 (85% chance of a 24.5-minute flooded trip) and C a utility of 71.2, so it picks C.
- Learning makes the same choices as Utility-Based, but from experience: in its rain context, B averaged 24.5 min and C 14.3 min over the training trips. It would need new training if the roads changed.
- Hierarchical splits the choice into layers. The strategic layer forbids congested and flood-prone roads, so the tactical layer is left with only B in Test 2 (C is slower) and only C in Test 3.
- Contextual recognises 'rush hour' and 'monsoon' and switches to the matching policy. It is right in all three tests, but only because these contexts were designed in advance.

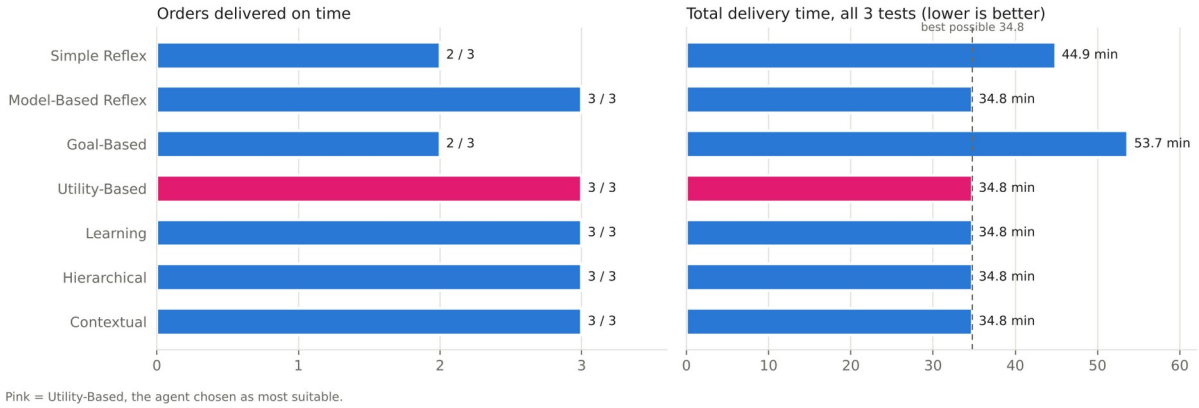


Figure: on-time deliveries and total delivery time per agent.

7.2 Feature comparison

Agent	Handles hidden flood	Handles traffic	Needs training	Design effort	Explains its choice
Simple Reflex	No	Yes (visible jam)	No	Very low	Rule that fired
Model-Based Reflex	Yes (rule)	Yes	No	Low	State + rule
Goal-Based	No	No	No	Low	Search path
Utility-Based	Yes (probability)	Yes	No	Medium	Utility of every route
Learning	Yes (learned)	Yes	Yes, 54 trips	Medium	Learned values
Hierarchical	Yes (constraint)	Yes	No	Medium	Each layer's decision
Contextual	Yes (context)	Yes	No	Medium	Context + policy

Five agents deliver all three orders on time in 34.8 minutes in total, so the results alone do not decide.

7.3 The most suitable agent

We choose the Utility-Based agent as the most suitable for this problem, for these reasons:

- It deals with uncertainty directly. The flood cannot be observed, and the agent reasons with P(flood) instead of a yes/no rule. A rule that is wrong 15% of the time is still a rule; the utility agent knows how risky B is.
- It balances several goals at once: being on time, total minutes and rider safety. The weights can be changed (for example, a heavier penalty for lateness on a premium order) without rewriting the agent.
- It needs no training data, unlike the Learning agent, and does not depend on a fixed list of contexts, unlike the Contextual agent. A new mix of conditions (for example a jam on A and light rain together) is still scored correctly.
- Its reasoning is transparent: the utility of every route is shown, which makes each decision easy to explain to the rider or the dispatcher.

The Learning agent is a strong second choice for a real company with a lot of trip data, and the Hierarchical design is a good way to add riding control on top of the Utility-Based route choice.

8. Conclusion

The simple architectures do well while the world is fully visible (Tests 1 and 2), but they fail when an important fact is hidden (Test 3). Agents that keep a model of the world, learn from experience or reason

about outcomes avoid the flooded underpass and deliver on time. The Utility-Based agent combines this with explicit handling of uncertainty and adjustable goals, so it is the most suitable architecture for the Smart Food Delivery Agent. The 3D simulation makes these differences visible: the same order, the same roads, and seven riders that think in seven different ways.